

3.1 MLIR 中的重写模式 与模式驱动

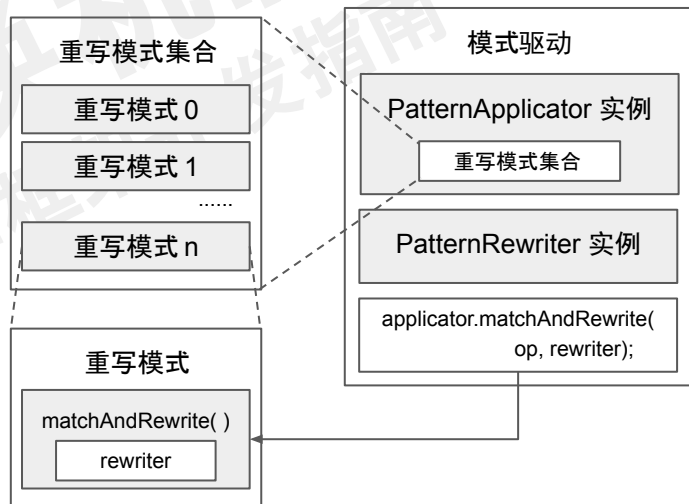
MLIR 中的重写模式与模式驱动

重写模式是对根操作执行模式匹配和 IR 重写的规则集合。

重写模式的核心功能是通过 `matchAndRewrite()` 方法对根操作执行模式匹配和 IR 重写。

根操作是某种重写模式主要针对的操作类型。例如，重写模式 `Add(a, a) -> Mul(a, 2)` 的根操作为 `Add`。

模式驱动是驱动 IR 重写过程的要素集合。



MLIR 中的重写模式与模式驱动

```
struct ConvertNVGPUToNVVMPass
: public impl::ConvertNVGPUToNVVMBase<ConvertNVGPUToNVVMPass> {
...
void runOnOperation() override {
    RewritePatternSet patterns(&getContext());
    LLVMTypeConverter converter(&getContext());

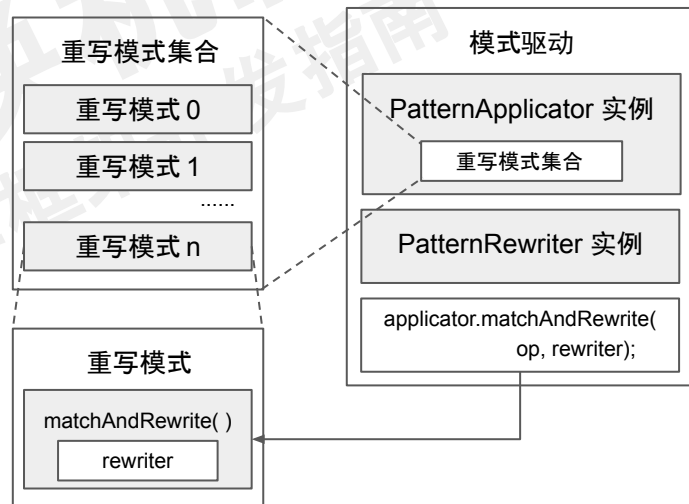
    ...
    populateNVGPUToNVVMConversionPatterns(converter, patterns);
    ...
    applyPartialConversion(...);
};

void mlir::populateNVGPUToNVVMConversionPatterns(
    LLVMTypeConverter &converter, RewritePatternSet &patterns) {
    patterns.add<MmaSyncOptoNVVM, MmaLdMatrixOpToNVVM, ...>(converter);
}

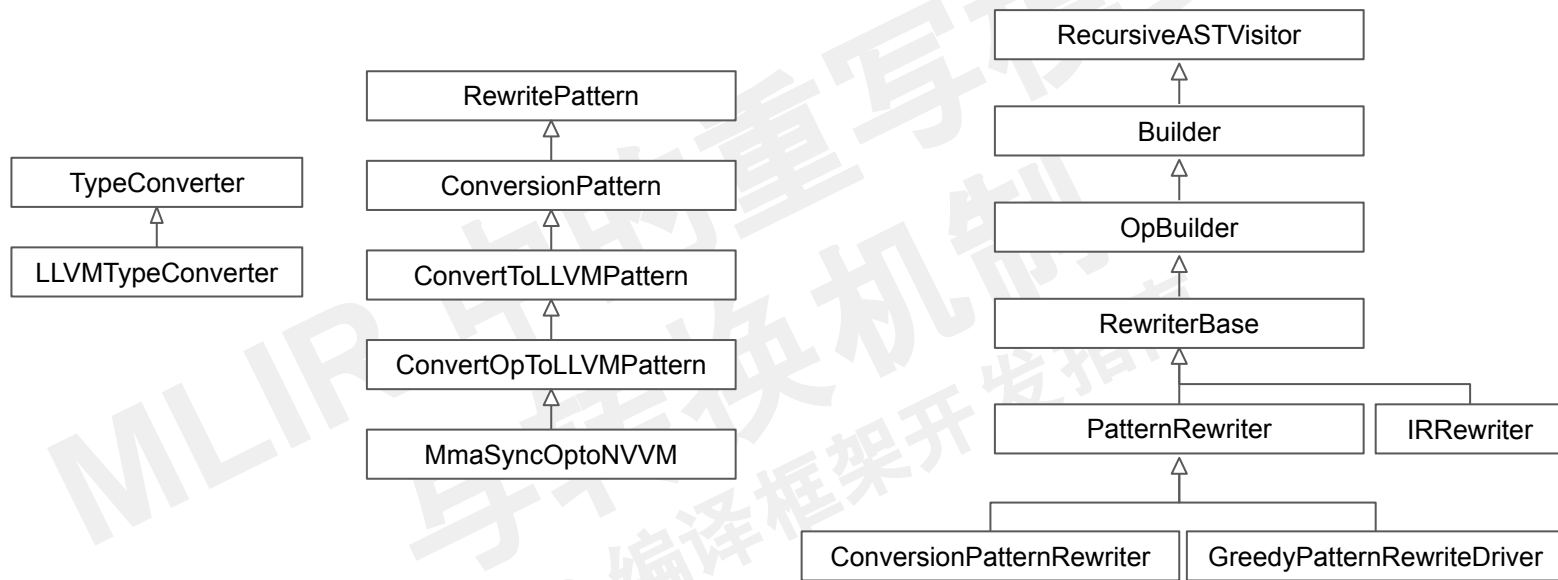
struct MmaSyncOptoNVVM : public
ConvertOpToLLVMPattern<nvgpu::MmaSyncOp> {
...
LogicalResult
matchAndRewrite(nvgpu::MmaSyncOp op, OpAdaptor adaptor,
    ConversionPatternRewriter &rewriter) const override {
    ...
    rewriter.replaceOp(op, rep);
    return success();
}
};

LogicalResult OperationConverter::convertOperations(
    ArrayRef<Operation *> ops,
    function_ref<void(Diagnostic &)> notifyCallback) {
    ...
    ConversionPatternRewriter rewriter(ops.front()->getContext());
    ...
    convert(rewriter, op);
}
```

```
LogicalResult
OperationLegalizer::legalizeWithPattern(Operation *op,
    ConversionPatternRewriter &rewriter) {
    ...
    return applicator.matchAndRewrite(...);
}
```



重写模式与类型转换器、重写器的类继承关系



类型转换器类继承关系

重写模式类继承关系

重写器类继承关系

通过继承 RewritePattern 类实现的重写模式称为基于 RewritePattern 规格的重写模式。

基于 TableGen 语言描述的重写模式

除了基于 RewritePattern 规格的重写模式外，还有基于 TableGen 语言描述的重写模式。

```
def : Pat<(GPU_BarrierOp), (NVVM_BarrierOp)>;
```

这种定义方式只是一种结构重写，不涉及数据 类型转换。

mlir-tblgen 生成的 .inc 文件中包含重写模式定义，可通过 populateWithGenerated() 函数添加到重写模式集合 patterns 中：

```
#include "GPToNVVM.cpp.inc"
```

```
...
```

```
void mlir::populateGpuToNVVMConversionPatterns(LLVMTypeConverter &converter,  
                                               RewritePatternSet &patterns) {
```

```
    populateWithGenerated(patterns);
```

```
    patterns.add<GPUPrintfOpToVPrintfLowering>(converter);
```

```
    patterns.add<GPUIndexIntrinsicOpLowering<gpu::ThreadIdOp, ...>,
```

```
    ...
```

驱动模式的组成要素

MLIR 提供了三种模式驱动:方言转换驱动、贪婪模式重写驱动和遍历模式重写驱动。

方言转换驱动的组成要素包括:类型转换器、重写模式集合、转换目标、模式重写器、执行转换接口
`apply*Conversion()`。

方言转换驱动将操作转换和类型转换分开。

贪婪模式重写驱动以工作列表驱动方式,将最大收益模式反复应用于 MLIR 操作,并选择使用局部收益最高的模式,直到达到最大迭代次数或迭代过程收敛。

贪婪模式重写驱动的组成要素中不包括:类型转换器和转换目标。

按照模式应用对象,贪婪模式重写驱动又可分为基于区域的驱动和基于操作的驱动。

